

05/12/2025

Proyecto Final

“Gestor del Festival FLORA Córdoba”

Desarrollo web entorno servidor

PILAR GIRÓN JORDÁN

2º DAW

ÍNDICE

ÍNDICE	1
Proyecto Final 1T: “Gestor del Festival FLORA Córdoba”	2
CREACIÓN DEL PROYECTO	2
MODELOS CREADOS Y SUS RELACIONES	3
VIEW	5
FILTRADO Y ORDENACIÓN	10
TEMPLATES y NAVEGACIÓN POR LA APLICACIÓN	10
INTEGRACIÓN DE DOCKER EN MI PROYECTO DE DJANGO	16
CONCLUSIÓN	20

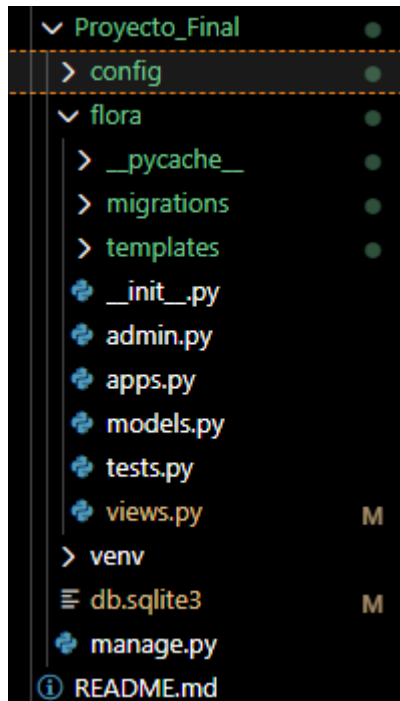
Proyecto Final 1T: “Gestor del Festival FLORA Córdoba”

Este proyecto final consiste en la elaboración de una aplicación web llamada “flora”, que se encarga de la gestión del Festival de Flora organizado en Córdoba.

El proyecto se ha desarrollado en Django y se apoya en templates de HTML para redireccionar a las diferentes secciones. La siguiente app que vamos a detallar a continuación permite por ejemplo, filtrar artistas por país, acceder al detalle de cada entidad y navegar fácilmente entre listados y formularios.

CREACIÓN DEL PROYECTO

En primer lugar hemos creado una carpeta llamada Proyecto_Final dentro de la cual se encuentra nuestra app llamada “flora”.



MODELOS CREADOS Y SUS RELACIONES

En nuestra raíz de proyecto dentro de `models.py` hemos creado los siguientes modelos, con los siguientes atributos.

Artist: name, country, short_bio y website.

Installation: title, opening_date, short_description y materials.

Venue: name, address, description y max_capacity.

Edition: year, theme, start_date, end_date.

Adjunto captura de pantalla.

```

1  from django.db import models
2
3  # Create your models here.
4  class Artist(models.Model):
5      name = models.CharField(max_length=200)
6      country= models.CharField(max_length=200)
7      short_bio= models.TextField()
8      website= models.URLField()
9
10     def __str__(self):
11         return self.name
12
13     class Venue(models.Model):
14         name = models.CharField(max_length=200)
15         address= models.CharField(max_length=200)
16         description= models.TextField()
17         max_capacity= models.IntegerField()
18
19         def __str__(self):
20             return self.name
21
22     class Edition(models.Model):
23         year= models.IntegerField()
24         theme= models.CharField(max_length=200)
25         start_date= models.DateField()
26         end_date= models.DateField()
27
28         def __str__(self):
29             return f"{self.year} - {self.theme}"
30
31     class Installation(models.Model):
32         title= models.CharField(max_length=200)
33         opening_date= models.DateField()
34         short_description= models.TextField()
35         materials= models.CharField(max_length=200)
36
  
```

En los modelos he establecido las siguientes relaciones:

Un artista puede tener varias obras (instalaciones). Cada obra se realiza en un único lugar (venue) y forma parte de una edición concreta del festival. Así, un lugar o una edición pueden tener muchas obras de distintos artistas.

```
#relaciones
artist = models.ForeignKey(Artist, on_delete=models.CASCADE, related_name='installations')
venue = models.ForeignKey(Venue, on_delete=models.CASCADE, related_name='installations')
edition = models.ForeignKey(Edition, on_delete=models.CASCADE, related_name='installations')
```

Para relaciones (ForeignKey) he usado `on_delete=models.CASCADE`, lo que garantiza que si se borra un artista, venue o edición, sus instalaciones asociadas también se eliminan.

VIEW

Después de crear los modelos, he creado las vistas en el archivo `view.py`

Estas vistas debían cumplir unas funcionalidad mínimas que detallo a continuación:

Artist: Listado, Detalle (incluyendo sus obras), Creación, Edición, Borrado, Filtrado por país.

```
Windsurf: Refactor | Explain | Generate Docstring | X
13 def artist_create(request):
14     if request.method == 'POST':
15         name = request.POST.get('name')
16         country = request.POST.get('country')
17         short_bio = request.POST.get('short_bio')
18         website = request.POST.get('website')
19
20         # Crear objeto Artist
21         artist = Artist(
22             name=name,
23             country=country,
24             short_bio=short_bio,
25             website=website
26         )
27         artist.save()
28
29         messages.success(request, 'Artista creado correctamente.')
30         return redirect('artist_list')
31     else:
32         return render(request, 'artist_create.html')
33
34
35
36 # Artist: Listado de artistas
Windsurf: Refactor | Explain | Generate Docstring | X
37 def artist_list(request):
38     country = request.GET.get('country', '').strip()
39     if country:
40         artists = Artist.objects.filter(country__icontains=country)
41     else:
42         artists = Artist.objects.all()
43     return render(request, 'artist_list.html', {'artists': artists, 'country': country})
44
45 #detalle incluyendo obras
Windsurf: Refactor | Explain | Generate Docstring | X
46 def artist_detail(request, artist_id):
47     artist = Artist.objects.get(id=artist_id)
48     return render(request, 'artist_detail.html', {'artist': artist})
49
50 #edición
Windsurf: Refactor | Explain | Generate Docstring | X
51 def artist_edit(request, artist_id):
52     artist = Artist.objects.get(id=artist_id)
53     if request.method == 'POST':
54         artist.name = request.POST.get('name')
55         artist.country = request.POST.get('country')
56         artist.short_bio = request.POST.get('short_bio')
57         artist.website = request.POST.get('website')
58         artist.save()
59         # Mensaje de éxito
60         messages.success(request, 'Artista editado correctamente.')
61
62         # Redirigir al listado de artistas
63         return redirect('artist_list')
64     else:
65         return render(request, 'artist_edit.html', {'artist': artist})
66
67 #borrado
Windsurf: Refactor | Explain | Generate Docstring | X
68 def artist_delete(request, artist_id):
69     artist = Artist.objects.get(id=artist_id)
70     artist.delete()
71     return redirect('artist_list')
72
73 #filtrado por pais
Windsurf: Refactor | Explain | Generate Docstring | X
74 def artist_list_by_country(request, country):
75     artists = Artist.objects.filter(country=country)
76     return render(request, 'artist_list.html', {'artists': artists})
77
```

Installation: Listado, Detalle (incluyendo el autor), Creación, Filtrado por edición.
Ordenar por fecha de inauguración (de forma ascendente o descendente).

```
#INSTALLATION

#listado
@staticmethod
def installation_list(request):
    installations = Installation.objects.all()
    data = {}
    for inst in installations:
        data.append({
            "id": inst.id,
            "title": inst.title,
            "opening_date": inst.opening_date.strftime("%Y-%m-%d"),
            "short_description": inst.short_description,
            "materials": inst.materials,
            "artist": {
                "id": inst.artist.id,
                "name": inst.artist.name
            },
            "venue": {
                "id": inst.venue.id,
                "name": inst.venue.name
            },
            "edition": {
                "id": inst.edition.id,
                "year": inst.edition.year,
                "theme": inst.edition.theme
            }
        })
    return JsonResponse(data, safe=False)

#detalle incluyendo autor
@staticmethod
def installation_detail(request, installation_id):
    installation = Installation.objects.get(id=installation_id)
    return render(request, 'installation_detail.html', {'installation': installation})

#crear inst
@staticmethod
def installation_create(request):
    if request.method == "POST":
        try:
            # se convierte el json a un objeto Python
            data = json.loads(request.body)
            title = data.get("title")
            opening_date = data.get("opening_date")
            short_description = data.get("short_description")
            materials = data.get("materials")
            artist_id = data.get("artist")
            venue_id = data.get("venue")
            edition_id = data.get("edition")

            artist = Artist.objects.get(id=artist_id)
            venue = Venue.objects.get(id=venue_id)
            edition = Edition.objects.get(id=edition_id)

            installation = Installation(
                title=title,
                opening_date=opening_date,
                short_description=short_description,
                materials=materials,
                artist=artist,
                venue=venue,
                edition=edition
            )
            installation.save()

            # se convierte el json a un objeto Python
            return JsonResponse({
                "id": installation.id,
                "title": installation.title,
                "artist": installation.artist.name,
                "venue": installation.venue.name,
                "edition": {
                    "id": edition.id,
                    "year": edition.year,
                    "theme": edition.theme
                }
            })
        except Exception as e:
            return JsonResponse({"error": str(e)}, status=400)

#filtrado por edición
@staticmethod
def installation_list_by_edition(request, edition_id):
    installations = Installation.objects.filter(edition_id=edition_id)
    data = {}
    for inst in installations:
        data.append({
            "id": inst.id,
            "title": inst.title,
            "opening_date": inst.opening_date.strftime("%Y-%m-%d"),
            "short_description": inst.short_description,
            "materials": inst.materials,
            "artist": {
                "id": inst.artist.id,
                "name": inst.artist.name
            },
            "venue": {
                "id": inst.venue.id,
                "name": inst.venue.name
            },
            "edition": {
                "id": inst.edition.id,
                "year": inst.edition.year,
                "theme": inst.edition.theme
            }
        })
    return JsonResponse(data, safe=False)

#ordenar por fecha de inauguración (de forma ascendente o descendente).
@staticmethod
def installation_list_by_opening_date(request, order):
    installations = Installation.objects.order_by('opening_date' if order == 'asc' else '-opening_date')
    data = {}
    for inst in installations:
        data.append({
            "id": inst.id,
            "title": inst.title,
            "opening_date": inst.opening_date.strftime("%Y-%m-%d"),
            "short_description": inst.short_description,
            "materials": inst.materials,
            "artist": {
                "id": inst.artist.id,
                "name": inst.artist.name
            },
            "venue": {
                "id": inst.venue.id,
                "name": inst.venue.name
            },
            "edition": {
                "id": inst.edition.id,
                "year": inst.edition.year,
                "theme": inst.edition.theme
            }
        })
    return JsonResponse(data, safe=False)
```

Venue: Listado, Detalle (incluyendo las obras expuestas), Creación.

```
213 #VENUE
214
215 #listado
216 def venue_list(request):
217     venues = Venue.objects.all()
218     data = []
219     for venue in venues:
220         data.append({
221             "id": venue.id,
222             "name": venue.name,
223             "address": venue.address,
224             "description": venue.description,
225             "max_capacity": venue.max_capacity
226         })
227
228     return JsonResponse(data, safe=False)
229
230
231 #detalle incluyendo obras
232 def venue_detail(request, venue_id):
233     venue = Venue.objects.get(id=venue_id)
234     try:
235         venue = Venue.objects.get(id=venue_id)
236         data = {
237             "id": venue.id,
238             "name": venue.name,
239             "address": venue.address,
240             "description": venue.description,
241             "max_capacity": venue.max_capacity
242         }
243         return JsonResponse(data)
244     except Venue.DoesNotExist:
245         return JsonResponse({"error": "Venue not found"}, status=404)
246
247 @csrf_exempt
248 #creacion
249 def venue_create(request):
250     if request.method == 'POST':
251         data = json.loads(request.body)
252         name = data.get('name')
253         address = data.get('address')
254         description = data.get('description')
255         max_capacity = data.get('max_capacity')
256         venue = Venue(name=name, address=address, description=description, max_capacity=max_capacity)
257         venue.save()
258         return JsonResponse({'id': venue.id, 'name': venue.name})
259
```

Edition (mínimo una acción): Listado, Detalle, Creación.

```
#EDICION

#listado
Windsurf: Refactor | Explain | Generate Docstring | X
def edition_list(request):
    editions = Edition.objects.all()
    data = []
    for edition in editions:
        data.append({
            "id": edition.id,
            "year": edition.year,
            "theme": edition.theme,
            "start_date": edition.start_date,
            "end_date": edition.end_date
        })

    return JsonResponse(data, safe=False)

#detalle incluyendo obras
Windsurf: Refactor | Explain | Generate Docstring | X
def edition_detail(request, edition_id):
    try:
        edition = Edition.objects.get(id=edition_id)
        data = {
            "id": edition.id,
            "year": edition.year,
            "theme": edition.theme,
            "start_date": edition.start_date,
            "end_date": edition.end_date
        }
        return JsonResponse(data)
    except Edition.DoesNotExist:
        return JsonResponse({"error": "Edition not found"}, status=404)

#creacion
Windsurf: Refactor | Explain | Generate Docstring | X
@csrf_exempt
def edition_create(request):
    if request.method == 'POST':
        try:
            # Leer JSON desde el body
            data = json.loads(request.body)
            year = data.get('year')
            theme = data.get('theme')
            start_date = data.get('start_date')
            end_date = data.get('end_date')

            edition = Edition(
                year=year,
                theme=theme,
                start_date=start_date,
                end_date=end_date
            )
            edition.save()

            return JsonResponse({
                "id": edition.id,
                "year": edition.year,
                "theme": edition.theme,
                "start_date": edition.start_date,
                "end_date": edition.end_date
            })

        except Exception as e:
            return JsonResponse({"error": str(e)}, status=400)

#PAGINA DE INICIO
Windsurf: Refactor | Explain | Generate Docstring | X
def home(request):
    return render(request, 'index.html')
```

Una vez creadas las vistas, he creado la relacion de URLs en `urls.py`

```
from django.contrib import admin
from django.urls import path
from flora import views

urlpatterns = [
    path('admin/', admin.site.urls),
    # ARTIST
    path('artists/', views.artist_list, name='artist_list'),
    path('artists/create/', views.artist_create, name='artist_create'),
    path('artists/<int:artist_id>/', views.artist_detail, name='artist_detail'),
    path('artists/<int:artist_id>/edit/', views.artist_edit, name='artist_edit'),
    path('artists/<int:artist_id>/delete/', views.artist_delete, name='artist_delete'),
    path('artists/country/<str:country>/', views.artist_list_by_country, name='artist_list_by_country'),
    # INSTALLATION
    path('installations/', views.installation_list, name='installation_list'),
    path('installations/create/', views.installation_create, name='installation_create'),
    path('installations/<int:installation_id>/', views.installation_detail, name='installation_detail'),
    path('installations/edition/<int:edition_id>/', views.installation_list_by_edition, name='installation_list_by_edition'),
    path('installations/order/<str:order>/', views.installation_list_by_opening_date, name='installation_list_by_opening_date'),
    # VENUE
    path('venues/', views.venue_list, name='venue_list'),
    path('venues/create/', views.venue_create, name='venue_create'),
    path('venues/<int:venue_id>/', views.venue_detail, name='venue_detail'),
    # EDITION
    path('editions/', views.edition_list, name='edition_list'),
    path('editions/create/', views.edition_create, name='edition_create'),
    path('editions/<int:edition_id>/', views.edition_detail, name='edition_detail'),
    #HOME
    path('', views.home, name='home'),
]
```

Importante incluir nuestra app en el archivo de setting.

```
1 # Application definition
2
3 INSTALLED_APPS = [
4     'django.contrib.admin',
5     'django.contrib.auth',
6     'django.contrib.contenttypes',
7     'django.contrib.sessions',
8     'django.contrib.messages',
9     'django.contrib.staticfiles',
10    'flora',
11 ]
```

FILTRADO Y ORDENACIÓN

En la aplicación Flora, el filtrado permite al usuario mostrar solo los registros que cumplen ciertos criterios, como el país de un artista. Por ejemplo:

En el listado de artistas, se puede escribir un país en el campo de filtrado y hacer clic en "Filtrar". La vista recibe ese parámetro desde la URL (`request.GET['country']`) y devuelve solo los artistas cuyo país coincide o contiene el valor ingresado.

```
#filtrado por país
Windsurf: Refactor | Explain | Generate Docstring | X
def artist_list_by_country(request, country):
    artists = Artist.objects.filter(country=country)
    return render(request, 'artist_list.html', {'artists': artists})
```

Listado de Artistas

Filtrar por país:

Nombre	País	Acciones
Jose Jimenez	España	Editar Borrar
Pedro Cano	España	Editar Borrar
Issac Marin	España	Editar Borrar

[Crear nuevo artista](#)

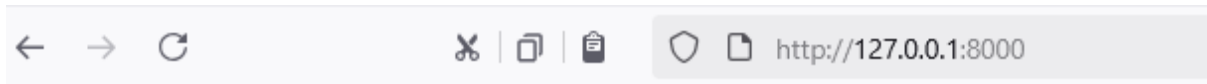
Aquí podemos observar un ejemplo de ordenación tanto ascendente como descendente de las fechas de inauguración.

```
#Ordenar por fecha de inauguración (de forma ascendente o descendente).
Windsurf: Refactor | Explain | Generate Docstring | X
def installation_list_by_opening_date(request, order):
    installations = Installation.objects.order_by('opening_date' if order == 'asc' else '-opening_date')
    return render(request, 'installation_list.html', {'installations': installations})
```

TEMPLATES y NAVEGACIÓN POR LA APLICACIÓN

Se ha creado una carpeta llamada templates dentro de la raíz del proyecto para incluir nuestra plantillas en html.

En primer lugar he creado una página de inicio o home en la que se listan enlaces a artistas, instalaciones, venues y ediciones.

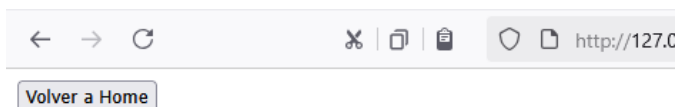


Gestor del Festival FLORA Córdoba

- [Listado de Artistas](#)
- [Listado de Instalaciones](#)
- [Listado de Venues](#)
- [Listado de Ediciones](#)

Dentro de Listado de Artista, nos mostrará los artistas existentes en nuestra base de datos o dará la opción de crear nuevos artistas. Además aquí hemos incluido la funcionalidad de filtrado por país, borrado o editado de artista.

Captura de filtrado



Listado de Artistas

Filtrar por país:

Nombre	País	Acciones
Rosa Carmona	Argentina	Editar Borrar

[Crear nuevo artista](#)

Indicar también que cuando entramos en el detalle de un artista hemos incluido dos botones para ir hacia el listado general o para la home nuevamente.

[Volver al listado](#) [Volver a Home](#)

Jose Jimenez

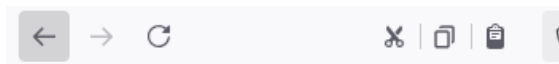
País: España

Biografía: Artista español especializado en esculturas

Web: <http://josejimenez.es>

[Editar artista](#) | [Borrar artista](#)

Captura de crear nuevo artista



Crear nuevo artista

Nombre:

País:

Biografía corta:

Sitio web:

[Volver](#) al listado de artistas.

Captura de editar artista

Editar Artista

Nombre:

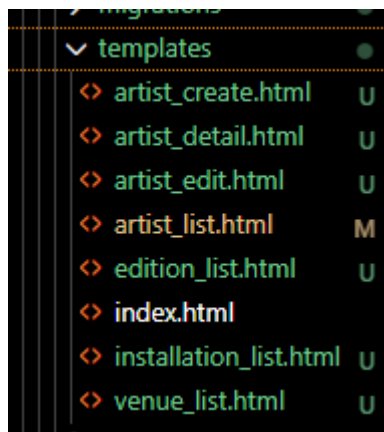
País:

Biografía corta:

Sitio web:

[Volver al listado de artistas](#)

En total hemos realizado contando con la home un total de 8 templates. Para las secciones de instalaciones, venues y ediciones, hemos realizado una plantilla estándar.

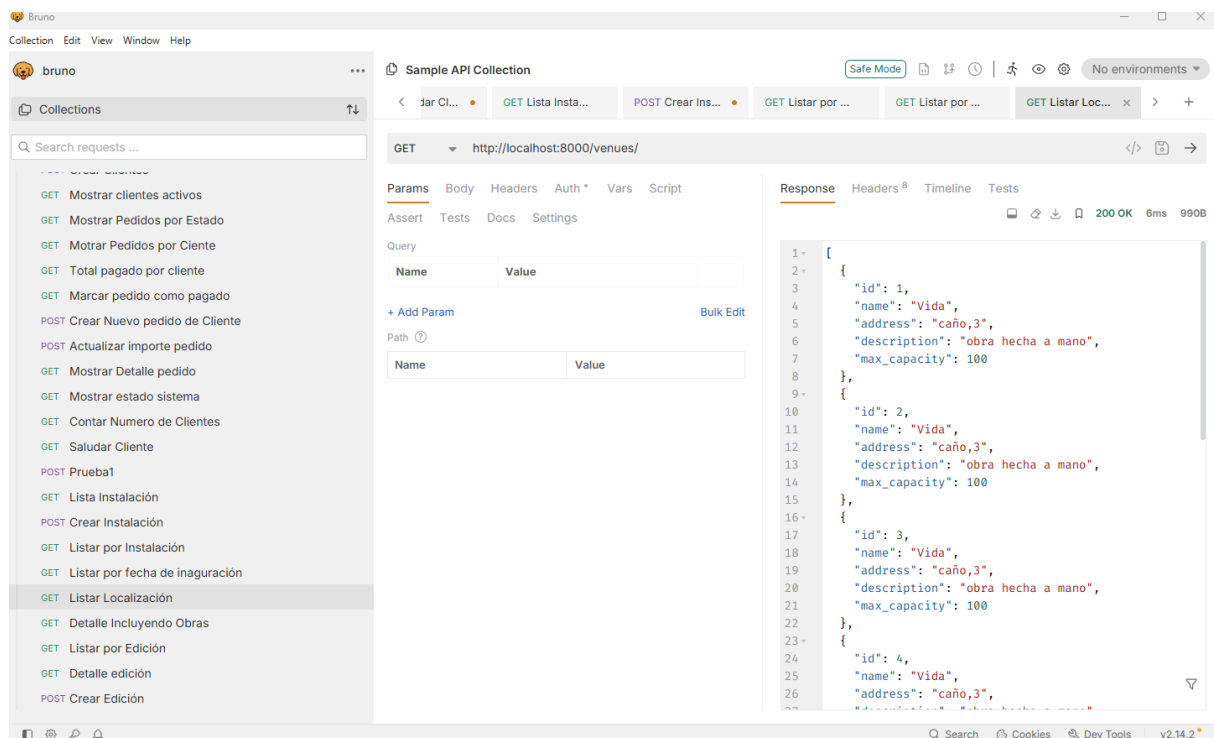
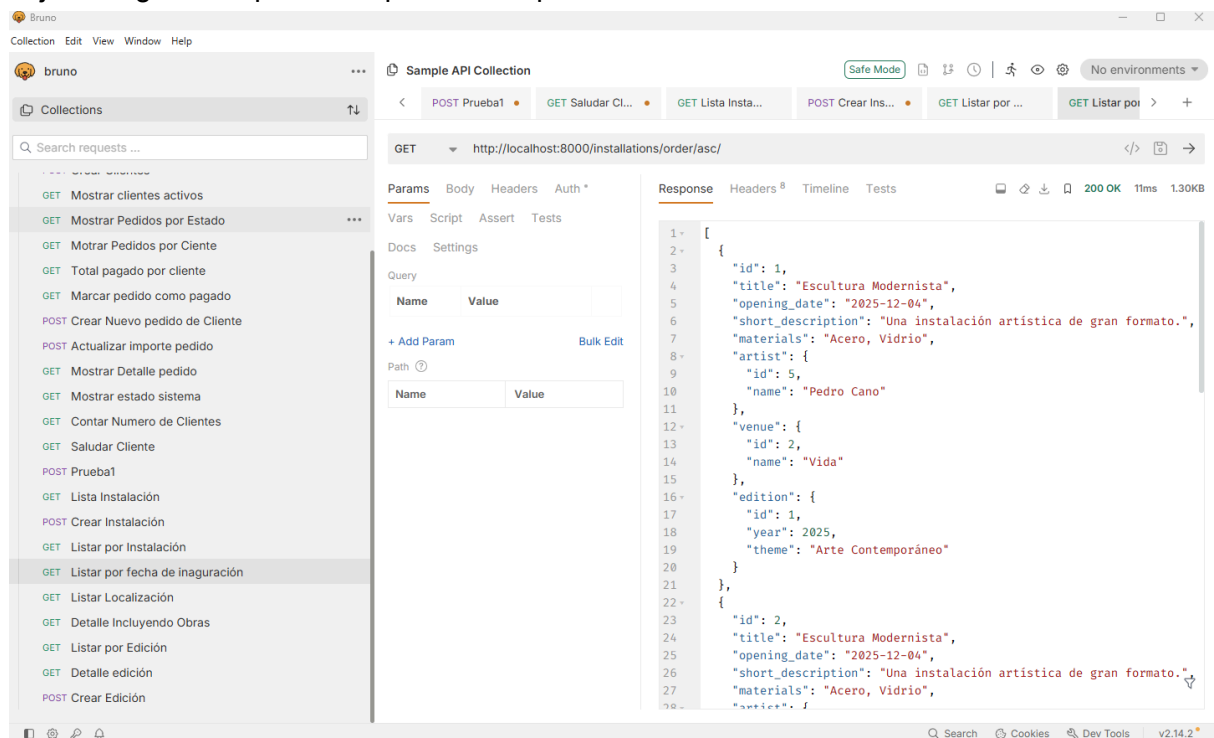


Sección de Instalaciones

Próximamente contenido.

[Volver a inicio](#)

Adjunto algunas capturas de pantalla de pruebas en Bruno



The screenshot shows the Bruno API client interface. On the left, a sidebar lists various API endpoints under the 'Sample API Collection'. The main panel displays a GET request to `http://localhost:8000/venues/2/`. The 'Response' tab is active, showing a JSON response with the following structure:

```
1 {
2   "id": 2,
3   "name": "Vida",
4   "address": "caño,3",
5   "description": "obra hecha a mano",
6   "max_capacity": 100
7 }
```

The status bar at the bottom indicates a 200 OK response with a 4ms duration and 108B of data.

The screenshot shows the Bruno API client interface. On the left, a sidebar lists various API endpoints under the 'Sample API Collection'. The main panel displays a POST request to `http://localhost:8000/editions/create/`. The 'Body' tab is active, showing a JSON request body with the following structure:

```
1 {
2   "year": 2024,
3   "theme": "Escultura Moderna",
4   "start_date": "2024-05-15",
5   "end_date": "2024-11-20"
6 }
```

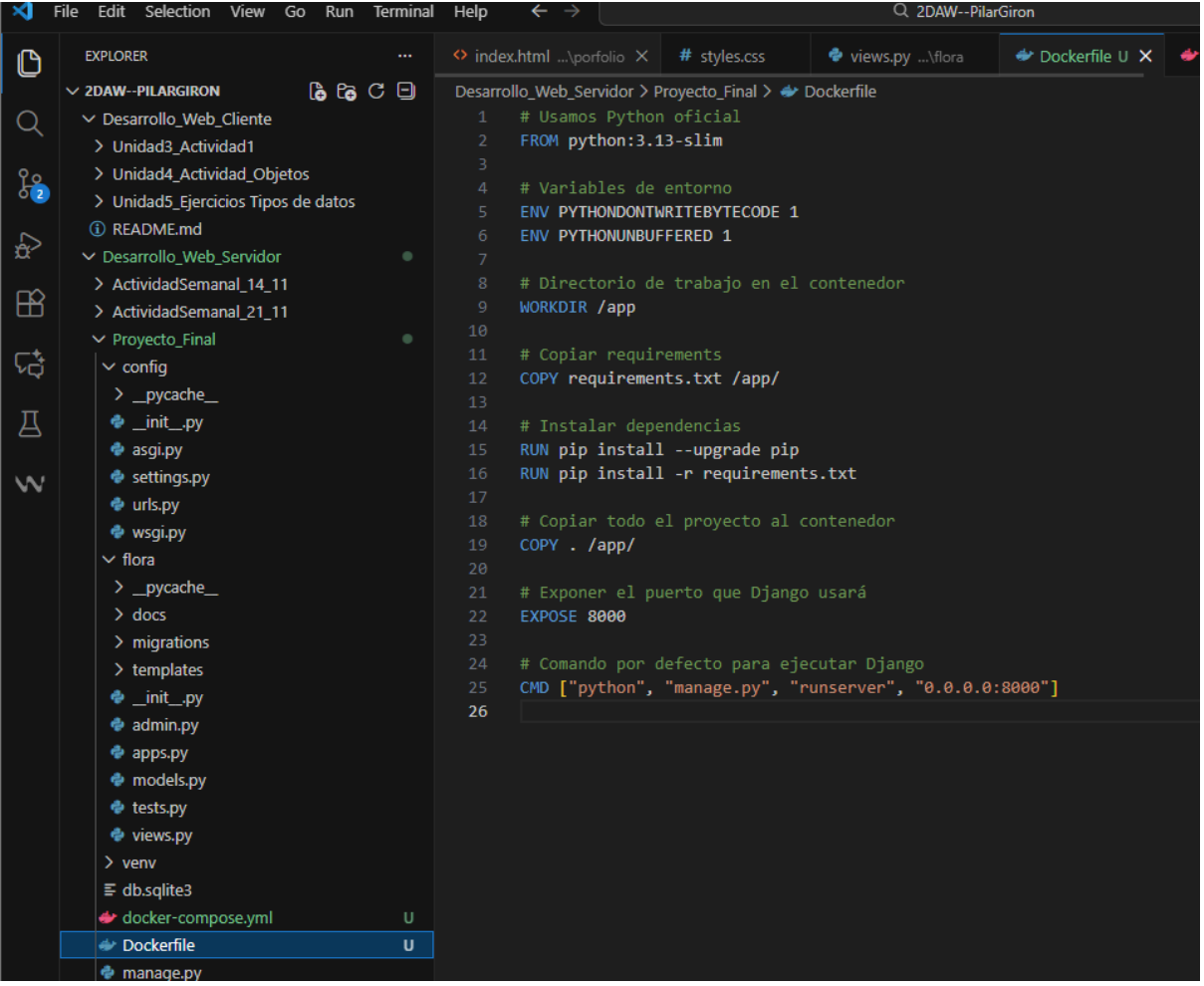
The 'Response' tab is also active, showing a JSON response with the following structure:

```
1 {
2   "id": 2,
3   "year": 2024,
4   "theme": "Escultura Moderna",
5   "start_date": "2024-05-15",
6   "end_date": "2024-11-20"
7 }
```

The status bar at the bottom indicates a 200 OK response with a 150ms duration and 107B of data.

INTEGRACIÓN DE DOCKER EN MI PROYECTO DE DJANGO

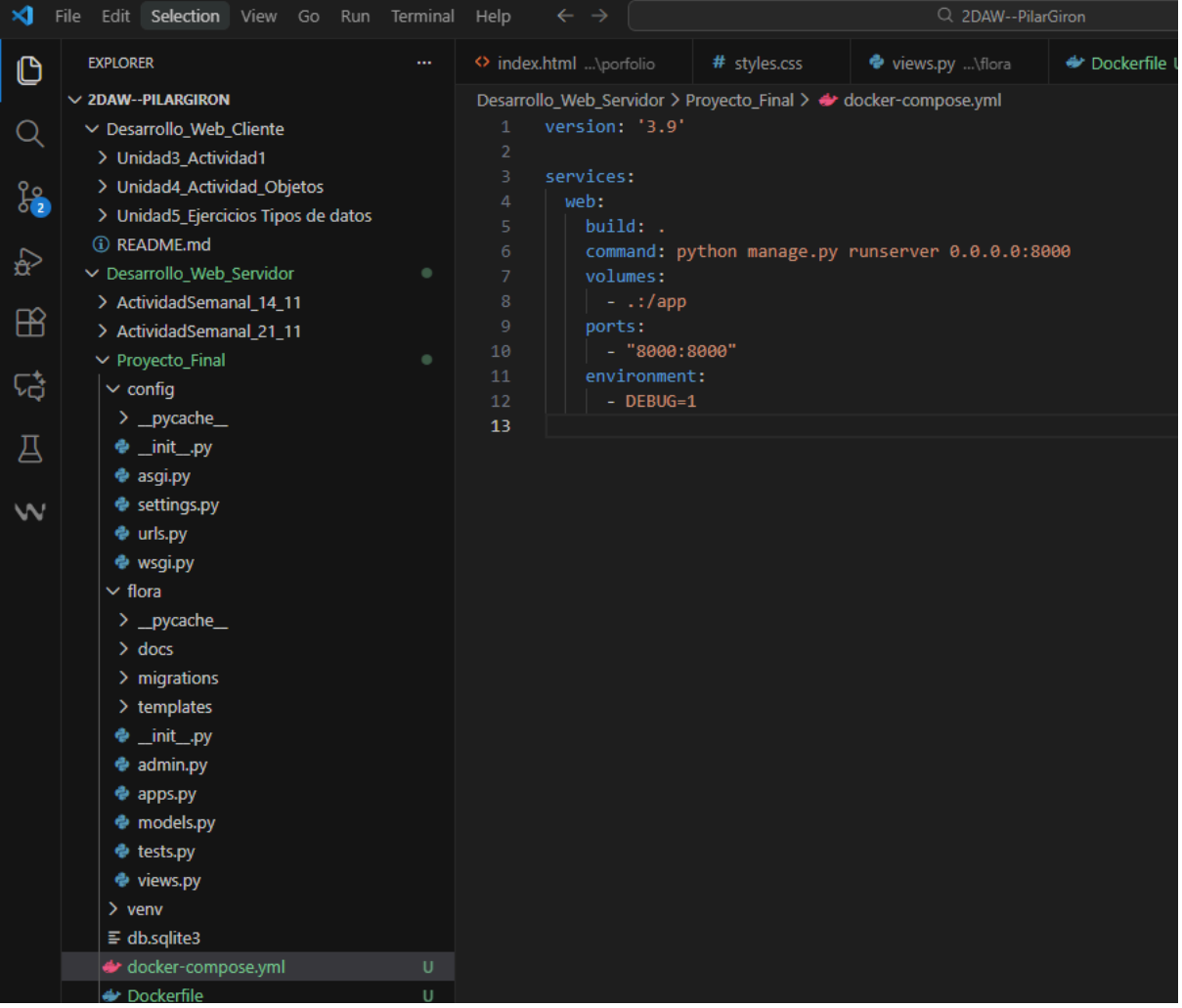
Creo archivo Dockerfile dentro de la raíz de mi proyecto.



The screenshot shows the Visual Studio Code interface with the Explorer panel on the left and the Dockerfile editor on the right. The Explorer panel shows the project structure for '2DAW--PILARGIRON', including folders for 'Desarrollo_Web_Cliente', 'Desarrollo_Web_Servidor', and 'Proyecto_Final'. The 'Proyecto_Final' folder is expanded, showing subfolders like 'config', 'flora', and 'venv', along with various Python files. The Dockerfile editor shows the following content:

```
1 # Usamos Python oficial
2 FROM python:3.13-slim
3
4 # Variables de entorno
5 ENV PYTHONDONTWRITEBYTECODE 1
6 ENV PYTHONUNBUFFERED 1
7
8 # Directorio de trabajo en el contenedor
9 WORKDIR /app
10
11 # Copiar requirements
12 COPY requirements.txt /app/
13
14 # Instalar dependencias
15 RUN pip install --upgrade pip
16 RUN pip install -r requirements.txt
17
18 # Copiar todo el proyecto al contenedor
19 COPY . /app/
20
21 # Exponer el puerto que Django usará
22 EXPOSE 8000
23
24 # Comando por defecto para ejecutar Django
25 CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
26
```

Creo archivo docker-compose.yml para conectar Django con la base de datos.



The screenshot shows the Visual Studio Code interface with the Explorer sidebar on the left and the Editor on the right. The Explorer sidebar displays a project structure for '2DAW--PILARGIRON'. The Editor shows the 'docker-compose.yml' file with the following content:

```
1 version: '3.9'
2
3 services:
4   web:
5     build: .
6     command: python manage.py runserver 0.0.0.0:8000
7     volumes:
8       - ../app
9     ports:
10      - "8000:8000"
11     environment:
12      - DEBUG=1
13
```

Construimos los contenedores y los ponemos en funcionamiento.

```
>>> exit()
now exiting InteractiveConsole...
>> nv) PS D:\2º DAW\2DAW--PilarGiron\Desarrollo_Web_Servidor\Proyecto_Final> docker build -t django-app .
[+] Building 27.1s (12/12) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 590B
=> [internal] load metadata for docker.io/library/python:3.13-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/6] FROM docker.io/library/python:3.13-slim@sha256:05b11ecc93ea09e30569706568fb251c71b77d2a3908d338b77
=> => resolve docker.io/library/python:3.13-slim@sha256:05b11ecc93ea09e30569706568fb251c71b77d2a3908d338b77
=> => sha256:13a347a59782d57000d9d8f542cae148af6b9027fdce630d1412401301eeca5f 250B / 250B
=> => sha256:c9e669a16e74d8022e55e0c6239a6b0898b1e7edea584946ed700da9ec3ca479 1.29MB / 1.29MB
=> => sha256:f2ec8236a2e7d9404af3e612305d3206a01b5f4062e924100374ef93dfa602f4 11.79MB / 11.79MB
=> => extracting sha256:c9e669a16e74d8022e55e0c6239a6b0898b1e7edea584946ed700da9ec3ca479
=> => extracting sha256:f2ec8236a2e7d9404af3e612305d3206a01b5f4062e924100374ef93dfa602f4
=> => extracting sha256:13a347a59782d57000d9d8f542cae148af6b9027fdce630d1412401301eeca5f
=> [internal] load build context
=> => transferring context: 98.83MB
=> [2/6] WORKDIR /app
```

```
- LegacyKeyValueFormat: "ENV key=value" should be used instead of legacy "ENV key value" format (line 5)
(venv) PS D:\2º DAW\2DAW--PilarGiron\Desarrollo_Web_Servidor\Proyecto_Final> docker run -p 8000:8000 django-app
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
December 04, 2025 - 10:09:18
Django version 6.0, using settings 'config.settings'
Starting development server at http://0.0.0.0:8000/
Quit the server with CONTROL-C.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/6.0/howto/deployment/
[04/Dec/2025 10:15:53] "GET / HTTP/1.1" 200 523
Not Found: /favicon.ico
[04/Dec/2025 10:15:53] "GET /favicon.ico HTTP/1.1" 404 5648
[04/Dec/2025 10:16:06] "GET /admin HTTP/1.1" 301 0
[04/Dec/2025 10:16:06] "GET /admin/ HTTP/1.1" 302 0
[04/Dec/2025 10:16:06] "GET /admin/login/?next=/admin/ HTTP/1.1" 200 4080
[04/Dec/2025 10:16:06] "GET /static/admin/css/dark_mode.css HTTP/1.1" 200 3538
[04/Dec/2025 10:16:06] "GET /static/admin/css/base.css HTTP/1.1" 200 23100
[04/Dec/2025 10:16:06] "GET /static/admin/js/theme.js HTTP/1.1" 200 1653
[04/Dec/2025 10:16:06] "GET /static/admin/css/login.css HTTP/1.1" 200 951
(venv) PS D:\2º DAW\2DAW--PilarGiron\Desarrollo_Web_Servidor\Proyecto_Final> docker run -p 8000:8000 django-app
Watching for file changes with StatReloader
Performing system checks...
```

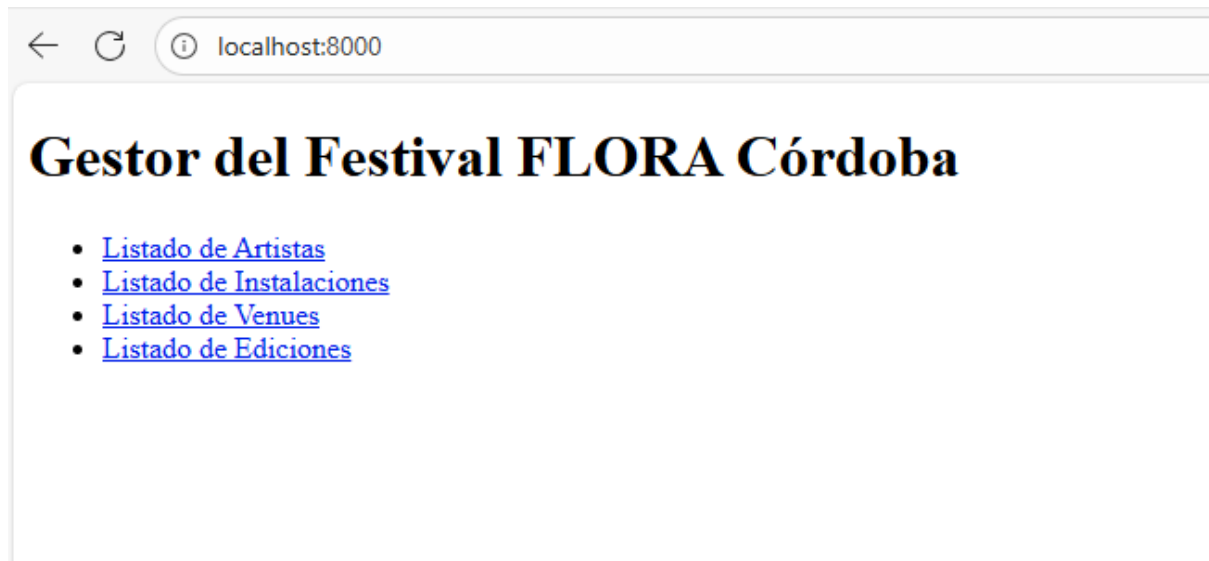
Obtenemos nuestro id de contenedor para luego acceder a él.

```
(venv) PS D:\2º DAW\2DAW--PilarGiron\Desarrollo_Web_Servidor\Proyecto_Final> docker ps
>>
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
43fe7da365f3   django-app    "python manage.py ru..." About a minute ago Up About a minute   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp   nostalgic_f
araday
```

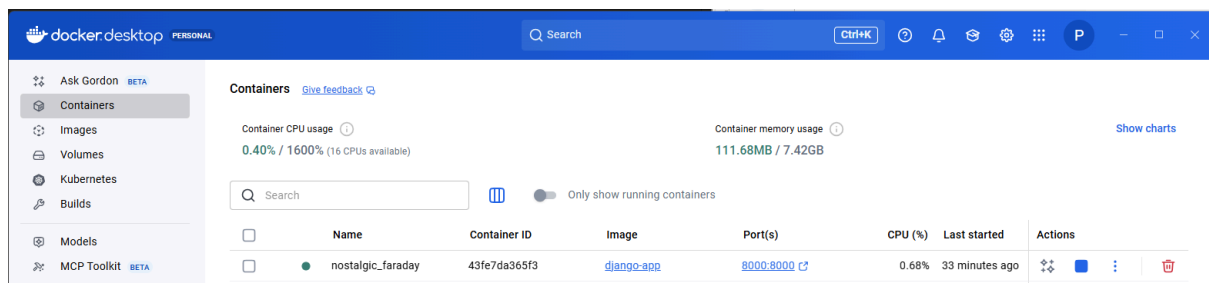
Accedemos al contenedor y aplicamos migraciones.

```
(venv) PS D:\2º DAW\2DAW--PilarGiron\Desarrollo_Web_Servidor\Proyecto_Final> docker exec -it 43fe7da365f3 bash
>>
root@43fe7da365f3:/app# python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, flora, sessions
Running migrations:
```

Probamos en nuestro navegador.



Comprobación en la app Desktop de Django la creación de nuestro contenedor para nuestra app flora.



CONCLUSIÓN

En este proyecto hemos creado la aplicación Flora, que permite gestionar artistas, obras, lugares y ediciones de un festival. Aprendimos a usar modelos para definir los datos y sus relaciones, vistas para mostrar, crear, editar y borrar registros (CRUD), y plantillas para que la información se vea en el navegador. También implementamos filtrado y ordenación para que los usuarios puedan encontrar lo que buscan fácilmente.